



## Travail Pratique Avancé :

« Conception, Déploiement et Sécurisation  
d'Architectures Web Modernes avec Supabase »

Auteur :  
BESNIARD Thomas

A destination de :  
BOURGUIBA Ak.

*Livré le : 22 octobre 2025*

## Sommaire

|                                    |         |
|------------------------------------|---------|
| Sommaire                           | Page 2  |
| I. Introduction                    | Page 3  |
| a. Présentation de SafeDocs        | Page 3  |
| b. Architecture Client-Serveur     | Page 3  |
| c. Architecture Micro-services API | Page 4  |
| d. Architecture Décentralisée      | Page 4  |
| II. Développement                  | Page 5  |
| a. Client-Serveur                  | Page 5  |
| b. Micro-services                  | Page 5  |
| c. Décentralisation                | Page 6  |
| III. Sécurité                      | Page 7  |
| a. Règles RLS                      | Page 7  |
| b. Token d'authentification        | Page 7  |
| c. Protection XSS                  | Page 8  |
| d. Reverse proxy                   | Page 8  |
| e. Failles détectées               | Page 9  |
| IV. Comparaison des architectures  | Page 10 |
| V. Conclusion                      | Page 11 |
| Annexes                            | Page 12 |

## I. Introduction

### a. Présentation de SafeDocs

Dans le cadre de ce travail pratique, nous avons étudié et mis en œuvre SafeDocs, un service web développé pour l'entreprise fictive DataSecure Corp. L'objectif de cette solution est de permettre aux utilisateurs de stocker, gérer et consulter leurs documents en ligne de manière simple et sécurisée.

SafeDocs se présente sous la forme d'une application web offrant plusieurs fonctionnalités essentielles : la création de compte utilisateur, la connexion sécurisée, le téléversement de fichiers personnels et leur consultation à travers une interface claire et accessible. Pour la gestion des données et du stockage, le projet repose sur Supabase, une plateforme cloud qui combine une base de données relationnelle et un système de fichiers. Ce choix permet de bénéficier d'une infrastructure fiable, performante et évolutive, tout en simplifiant la mise en place des services nécessaires à l'application.

### b. Architecture Client-Serveur

L'architecture client-serveur est l'un des modèles fondamentaux du développement d'applications web. Elle repose sur une séparation claire entre deux entités : le client, qui représente l'interface utilisée par l'utilisateur (souvent un navigateur web ou une application), et le serveur, qui héberge les données, la logique de traitement et les ressources nécessaires au fonctionnement de l'application. Le client envoie des requêtes au serveur (par exemple pour récupérer ou envoyer des données), et le serveur répond en fournissant les informations demandées.

Ce modèle présente plusieurs avantages : il facilite la centralisation des données, simplifie la maintenance du système, et permet à plusieurs utilisateurs d'accéder simultanément aux mêmes ressources. Cependant, il peut aussi montrer des limites en termes de scalabilité (montée en charge) ou de résilience, car le serveur constitue souvent un point unique de défaillance. Ces limites ont conduit à l'émergence de modèles plus distribués, comme les micro-services ou les architectures décentralisées.

### c. Architecture Micro-services API

L'architecture micro-services est une approche moderne du développement d'applications web, qui vise à diviser un système complexe en plusieurs services indépendants. Chaque micro-service est conçu pour remplir une fonction précise (comme la gestion des utilisateurs, le stockage des fichiers ou l'authentification) et communique avec les autres via des API (Interfaces de Programmation d'Applications).

Cette organisation offre une grande flexibilité : chaque service peut être développé, déployé ou mis à jour séparément, sans impacter le reste de l'application. Elle améliore également la scalabilité, car il est possible d'adapter les ressources allouées à un seul service selon la charge qu'il supporte. En revanche, cette approche demande une meilleure coordination entre les composants et une gestion plus rigoureuse de la communication et de la sécurité entre les services. Dans le cadre de SafeDocs, cette architecture permettrait par exemple de séparer la logique liée à l'authentification, à la gestion des fichiers et à la base de données, pour obtenir un système plus modulaire et résilient.

### d. Architecture Décentralisée

L'architecture décentralisée repose sur le principe de répartir le stockage et le traitement des données entre plusieurs nœuds indépendants, plutôt que de les concentrer sur un serveur unique. Contrairement au modèle client-serveur classique, où toutes les requêtes passent par un point central, une application décentralisée (souvent appelée DApp) s'appuie sur un réseau distribué dans lequel chaque participant peut agir à la fois comme client et comme serveur.

Ce type d'architecture apporte plusieurs avantages, notamment une meilleure résilience face aux pannes, une réduction des risques de censure et une plus grande transparence dans la gestion des données. En revanche, elle est souvent plus complexe à mettre en œuvre et à sécuriser. Dans le cadre du projet SafeDocs, une approche décentralisée pourrait permettre aux utilisateurs de conserver la maîtrise totale de leurs documents, en les stockant sur un réseau distribué plutôt que sur une infrastructure centralisée. Cela ouvrirait la voie à des solutions plus sûres et plus respectueuses de la vie privée.

## II. Développement

### a. Client-Serveur

Dans un premier temps, le développement de SafeDocs a été réalisé selon une architecture client-serveur classique. L'application repose sur un client web, chargé d'interagir avec l'utilisateur, et un serveur central qui gère la logique de traitement ainsi que la communication avec Supabase.

Toutes les interactions de l'utilisateur (création de compte, la connexion ou le téléversement de fichiers) sont gérées à travers des formulaires et des event listeners. Ces derniers permettent de déclencher les fonctions associées à chaque action. Cette approche rend le fonctionnement du client fluide et réactif, tout en limitant la complexité du backend (se référer aux annexes).

En effet, le serveur agit ici comme seul point central de communication : il reçoit les requêtes du client, les traite et renvoie les réponses correspondantes. Ce modèle s'est révélé être la solution la plus simple à mettre en place, car il nécessite moins de fonctionnalités côté serveur, tout en assurant une bonne séparation entre l'interface utilisateur et la logique applicative.

### b. Micro-Services

La mise en œuvre de SafeDocs selon une architecture à micro-services a demandé un travail bien plus conséquent que l'approche client-serveur. Alors que la première version avait été développée en seulement trois heures, cette seconde approche a nécessité près d'un jour et demi de travail. Cette différence s'explique par la complexité technique et la multiplicité des composants impliqués (se référer aux annexes).

En effet, une architecture à micro-services repose sur la création de plusieurs services indépendants, communiquant entre eux via une API REST. Pour que l'ensemble fonctionne correctement, il a été nécessaire de mettre en place des tokens d'authentification, de conteneuriser les services avec Docker, et d'ajouter un reverse proxy afin d'assurer la communication et la redirection entre les différentes parties du système.

Ce projet a représenté un véritable défi technique, car je n'avais encore jamais relié une API REST à une interface graphique, ni utilisé Docker ou un reverse proxy auparavant. J'ai donc dû me former en parallèle du développement, ce qui a rallongé le temps de mise en place mais m'a permis de mieux comprendre la partie système du déploiement web. Cette approche m'a beaucoup plu, car elle m'a offert une vision plus complète et concrète des enjeux liés à la modularité, la sécurité et la maintenance d'une application web moderne.

### c. Décentralisation

L'approche décentralisée n'a pas été mise en œuvre dans le cadre de ce travail pratique, principalement par manque de temps et de compétences techniques sur le sujet. Ce type d'architecture, plus complexe à déployer, repose sur des principes et des outils spécifiques que je n'ai pas encore eu l'occasion d'étudier en détail.

N'ayant jamais été formé à ce type de conception, il m'aurait été difficile d'en assurer la mise en place correcte dans le temps imparti. Cependant, cette partie du projet a éveillé ma curiosité : je compte acquérir les compétences nécessaires ultérieurement afin de pouvoir expérimenter ce modèle à l'avenir et prendre ma revanche sur cette étape du travail. Explorer une architecture décentralisée serait une opportunité d'élargir mes connaissances et de comprendre comment rendre les applications encore plus autonomes, sécurisées et résilientes.

### III. Sécurité

#### a. Règles RLS

Les RLS (Row-Level Security) sont un mécanisme de sécurité permettant de contrôler l'accès aux données au niveau de chaque ligne d'une table dans une base de données. Contrairement aux permissions globales, les RLS permettent de définir des règles très fines, par exemple pour qu'un utilisateur puisse uniquement accéder à ses propres données.

Dans SafeDocs, j'ai commencé par créer dans Supabase une table docs permettant de relier chaque utilisateur aux fichiers qu'il dépose. J'ai ensuite créé un bucket docBucket pour stocker les fichiers eux-mêmes. Pour sécuriser l'accès, j'ai défini des politiques RLS à la fois sur la table docs et sur le bucket docBucket. Ces règles assurent que seul l'utilisateur connecté peut déposer des fichiers et, surtout, que chaque utilisateur ne peut consulter que ses propres fichiers (se référer aux annexes).

Cette approche a été essentielle pour garantir la confidentialité des documents des utilisateurs et constitue une première couche de sécurité robuste côté base de données et stockage.

#### b. Token d'authentification

Un token d'authentification est une chaîne de caractères unique qui permet de vérifier l'identité d'un utilisateur lors de ses interactions avec une application ou une API. Contrairement à une simple session, un token peut être envoyé à chaque requête, assurant que chaque appel provient bien de l'utilisateur légitime.

Dans SafeDocs, cette fonctionnalité a été mise en place dans l'approche micro-services, afin de vérifier l'identité de l'utilisateur à chaque appel API. Le token joue un rôle essentiel à deux niveaux : d'une part, sur le plan sécuritaire, il empêche tout accès non autorisé aux services et aux données ; d'autre part, sur le plan fonctionnel, il permet d'identifier précisément l'utilisateur qui interagit avec SafeDocs, garantissant que chaque action (comme le dépôt ou la consultation d'un fichier) est correctement associée à son propriétaire.

### c. Protection XSS

Une attaque XSS (Cross-Site Scripting) consiste à injecter du code malveillant (souvent du JavaScript) dans une application web, afin de voler des données, détourner des sessions ou modifier le contenu affiché aux utilisateurs. Ce type de vulnérabilité est particulièrement critique pour les applications manipulant des données personnelles ou sensibles, comme SafeDocs.

Pour protéger l'application contre ces attaques, une fonction utilitaire a été mise en place dans chaque version livrée. Cette fonction applique une expression régulière (REGEX) sur un champ passé en paramètre afin de détecter toute tentative d'injection. Si le contenu semble suspect, l'action est bloquée, garantissant que seules des entrées sûres peuvent être traitées par l'application (se référer aux annexes).

Cette approche simple mais efficace permet de prévenir les injections XSS tout en restant légère et facile à intégrer dans toutes les parties de l'application manipulant des données utilisateur.

### d. Reverse proxy

Un reverse proxy est un serveur intermédiaire qui reçoit les requêtes des clients avant de les transmettre aux serveurs internes d'une application. Il joue plusieurs rôles : protéger les services internes, répartir la charge entre plusieurs serveurs, mettre en cache certaines réponses et simplifier la gestion des certificats SSL.

Dans l'approche micro-services de SafeDocs, j'ai mis en place un reverse proxy pour protéger mes deux conteneurs de services, auth-service et files-service, contre tout accès non autorisé. Pour ce faire, j'ai utilisé Nginx, une technologie que je ne connaissais pas avant ce travail pratique. Le reverse proxy a ainsi permis de centraliser les points d'entrée vers les services, d'ajouter une couche de sécurité et de simplifier la communication entre le client et les différents micro-services (se référer aux annexes).

Cette expérience m'a permis de découvrir le rôle crucial d'un reverse proxy dans la sécurisation et le déploiement d'architectures complexes, et de me familiariser avec une technologie largement utilisée dans l'industrie.



### e. Failles détectées

Après avoir réalisé des tests de sécurité, j'ai remarqué que les informations relatives à l'utilisateur (identifiants, token, courriel, téléphone...) sont placées directement dans un script exécuté côté client, elles deviennent accessibles à tout script de la page et même à la console du navigateur (`console.log`). Cela crée une fuite d'information : un attaquant qui réussit à injecter du code ou un simple utilisateur curieux peut lire ces données.

Afin de corriger cette fuite, il faudrait stocker ses informations du côté serveur, dans un fichier json par exemple.

## IV. Comparaison des architectures

| Critère                       | Client-Serveur                                                                                                                                                                                | Micro-services                                                                                                                                                                                                            | Décentralisée                                                                                                                                                                                                                     |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sécurité</b>               | Relativement simple à sécuriser grâce à un serveur central ; RLS et tokens suffisent pour protéger les données. Toutefois, un point de défaillance unique peut exposer l'ensemble du système. | Très sécurisable si chaque service est correctement isolé et que l'authentification/autorisation est bien gérée. La complexité de la communication entre services peut introduire de nouvelles failles si mal configurée. | Potentiellement très sécurisée car les données sont réparties et contrôlées par chaque nœud. La complexité de sécurisation augmente fortement, notamment pour le chiffrement et la gestion des identités sur un réseau distribué. |
| <b>Fiabilité</b>              | Dépend fortement du serveur central : si le serveur tombe, l'application entière est indisponible.                                                                                            | Plus fiable grâce à la séparation en services indépendants : une panne sur un service peut être isolée.                                                                                                                   | Très fiable car le réseau distribué peut continuer à fonctionner même si plusieurs nœuds tombent.                                                                                                                                 |
| <b>Scalabilité</b>            | Limitée : le serveur central peut devenir un goulot d'étranglement.                                                                                                                           | Très scalable : chaque service peut être mis à l'échelle indépendamment selon la charge.                                                                                                                                  | Très scalable naturellement grâce à la distribution des ressources entre de nombreux nœuds.                                                                                                                                       |
| <b>Temps de développement</b> | Court : simplicité du modèle et centralisation des fonctionnalités.                                                                                                                           | Long : nécessite plus de temps pour développer, conteneuriser et sécuriser chaque service, mettre en place les API et le reverse proxy.                                                                                   | Très long : requiert des compétences avancées, mise en place complexe et expérimentale si non familière.                                                                                                                          |
| <b>Temps de maintenance</b>   | Relativement faible : tout est centralisé, mais chaque modification peut impacter l'ensemble de l'application.                                                                                | Modéré à élevé : les services sont indépendants, donc certains changements peuvent être isolés, mais la coordination entre services demande une bonne gestion.                                                            | Potentiellement élevé : chaque nœud ou service doit être surveillé, mis à jour et sécurisé, ce qui augmente la complexité de maintenance.                                                                                         |

## V. Conclusion

Le choix de l'architecture d'une application web dépend fortement du contexte du projet, de ses objectifs et de ses contraintes.

Pour un projet comme SafeDocs, destiné à un prototype ou à une petite application avec un temps de développement limité, l'architecture client-serveur est la solution la plus adaptée. Elle permet de mettre rapidement en place les fonctionnalités essentielles, de sécuriser les données avec RLS et tokens, et de limiter la complexité technique.

Pour des projets plus ambitieux, nécessitant une scalabilité importante, une modularité avancée et une maintenance facilitée, l'architecture micro-services est préférable. Elle demande plus de temps et de compétences pour le développement et le déploiement, mais offre une meilleure résilience et une plus grande flexibilité pour évoluer au fil du temps.

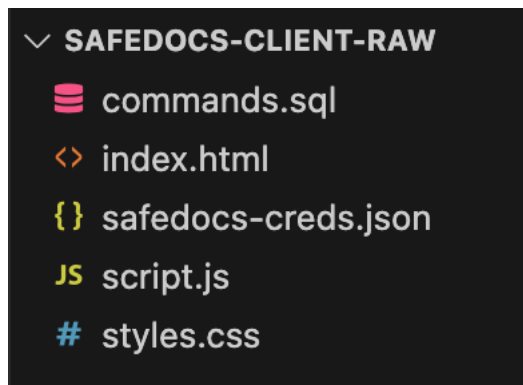
Enfin, une architecture décentralisée est intéressante dans des contextes où la sécurité et la résilience maximales sont prioritaires, par exemple pour des applications distribuées ou critiques où chaque utilisateur doit contrôler ses données. Cependant, elle nécessite des compétences avancées et un investissement important en temps de développement et de maintenance.

## Annexes

### Annexe 1 : Lien github du projet

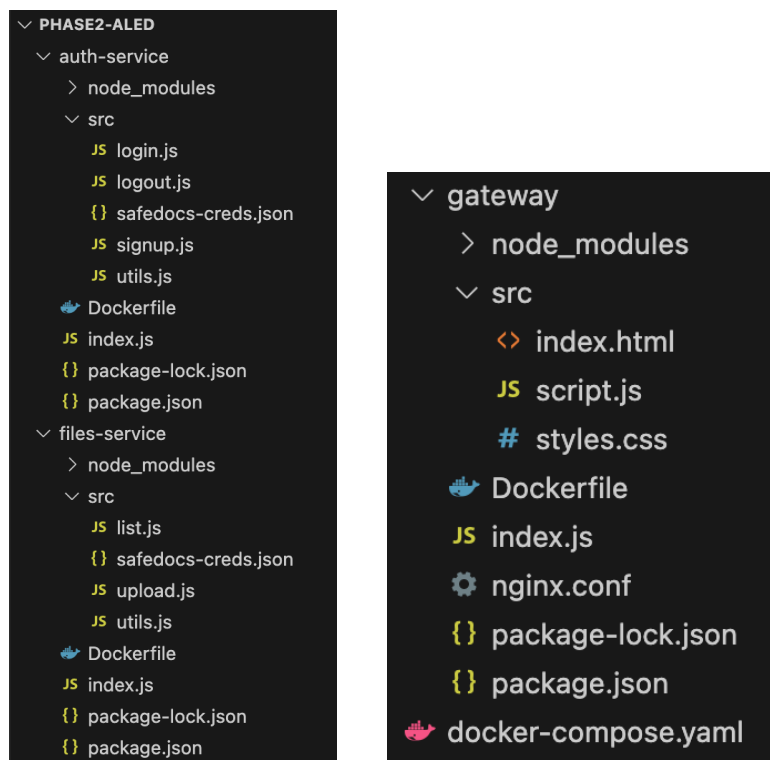
[https://github.com/Thr0nm41/TP\\_ArchitectureWeb](https://github.com/Thr0nm41/TP_ArchitectureWeb)

### Annexe 2 : Architecture Client-Serveur



Cette architecture nécessite très peu de fonctionnalités pour fonctionner (cette version ne pèse que 16 Ko)

### Annexe 3 : Architecture Micro-services



Cette architecture est beaucoup plus lourde : 33 Mo

## Annexe 4 : Token d'authentification stocké dans les informations utilisateur

```
{id: '815781be-1992-48b6-9606-69f014351696', aud: 'authenticated', role: 'authenticated', email: 'thronm41l@gmail.co
m', email_confirmed_at: '2025-10-20T12:06:09.394213Z', ...}
  access_token: "eyJhbGciOiJIUzI1NiIsImtpZCI6I1ZSunlHRlJ4ZnVuc2hEZEoiLCJ0eXAiOiJKV1QiLCJpc3MiOiJodHRwczovL3htbGpnYn
  app_metadata: {provider: 'email', providers: Array(1)}
  aud: "authenticated"
  confirmation_sent_at: "2025-10-20T12:03:02.395904Z"
  confirmed_at: "2025-10-20T12:06:09.394213Z"
  created_at: "2025-10-20T12:03:02.35762Z"
  email: "thronm41l@gmail.com"
  email_confirmed_at: "2025-10-20T12:06:09.394213Z"
  id: "815781be-1992-48b6-9606-69f014351696"
  identities: [{...}]
  is_anonymous: false
  last_sign_in_at: "2025-10-22T21:15:12.168583081Z"
  phone: ""
  role: "authenticated"
  updated_at: "2025-10-22T21:15:12.233374Z"
  user_metadata:
    email: "thronm41l@gmail.com"
    email_verified: true
    phone_verified: false
    sub: "815781be-1992-48b6-9606-69f014351696"
  [[Prototype]]: Object
  [[Prototype]]: Object
```

## Annexe 5 : Conteneurisation

| <input type="checkbox"/> | Name          | Container ID | Image                      | Port(s)   | CPU (%) | Actions                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|---------------|--------------|----------------------------|-----------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | phase2-aled   | -            | -                          | -         | 1.39%   |     |
| <input type="checkbox"/> | auth-service  | 49bdf988c827 | <a href="#">phase2-ale</a> |           | 1.39%   |     |
| <input type="checkbox"/> | files-service | 4b2da0e70dcb | <a href="#">phase2-ale</a> |           | 0%      |     |
| <input type="checkbox"/> | gateway       | a1985e3b051d | <a href="#">phase2-ale</a> | 3000:80 ↗ | 0%      |     |

## Annexe 6 : Policies RLS

```
-- Allow users to view (select) their own documents
create policy "Users can view their own documents"
on docs for select
using (auth.uid() = user_id);

-- Allow users to insert (create) documents for themselves
create policy "Users can insert their own documents"
on docs for insert
with check (auth.uid() = user_id);

-- Allow users to insert (upload) files into their own folder 1glxqgy_0
CREATE POLICY "Users can upload their own files"
ON storage.objects
FOR INSERT
WITH CHECK (
  bucket_id = 'docBucket'
  AND (storage.foldername(name))[1] = auth.uid()::text
);

-- Allow users to view (download) their own files from their folder 1glxqgy_0
CREATE POLICY "Users can download their own files"
ON storage.objects
FOR SELECT
USING (
  bucket_id = 'docBucket'
  AND (storage.foldername(name))[1] = auth.uid()::text
);
```

## Annexe 7 : Protection contre les attaques XSS

```
// Check input validity (email format, empty input, injection attempts)
function checkInputValidity(input, type="text") {
    const value = input.trim()
    // Basic validation for email format
    if (type === "email") {
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/ // Regex for email validation
        if(!emailRegex.test(value)) {
            afficherMessage("Please enter a valid email address.", "error")
            return false
        }
    }else{
        // Check for empty input
        if (value === "") {
            afficherMessage("This field cannot be empty.", "error")
            return false
        }
    }
    // Check for injection attempts
    const forbiddenChars = /[<'"\"\\\/\;\:]/ // Regex for forbidden characters
    if (forbiddenChars.test(value)) {
        afficherMessage("Input contains forbidden characters.", "error")
        return false
    }
    return true
}
```

## Annexe 8 : Reverse proxy

```
gateway > nginx.conf
1  server {
2      listen 80;
3      server_name localhost;
4
5      # Serve static files
6      location / {
7          root /usr/share/nginx/html;
8          try_files $uri $uri/ /index.html;
9      }
10
11     # Proxy API requests for auth-service
12     location /api/auth/ {
13         proxy_pass http://auth-service:3001/;
14         proxy_set_header Host $host;
15         proxy_set_header X-Real-IP $remote_addr;
16         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
17         proxy_set_header X-Forwarded-Proto $scheme;
18     }
19     # Support requests without trailing slash
20     location = /api/auth {
21         proxy_pass http://auth-service:3001/;
22         proxy_set_header Host $host;
23         proxy_set_header X-Real-IP $remote_addr;
24         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
25         proxy_set_header X-Forwarded-Proto $scheme;
26     }
27
28     # Proxy API requests for files-service
29     location /api/files/ {
30         proxy_pass http://files-service:3002/;
31         proxy_set_header Host $host;
32         proxy_set_header X-Real-IP $remote_addr;
33         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
34         proxy_set_header X-Forwarded-Proto $scheme;
35     }
36     # Support requests without trailing slash
37     location = /api/files {
38         proxy_pass http://files-service:3002/;
39         proxy_set_header Host $host;
40         proxy_set_header X-Real-IP $remote_addr;
41         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
42         proxy_set_header X-Forwarded-Proto $scheme;
43     }
44
45     # Optional: increase client body size for file uploads
46     client_max_body_size 50M;
47 }
48
```

## Annexe 9 : Structure de Supabase

*Supabase table docs (fait référence à la table par défaut auth.users) :*

Update table docs

Columns [About data types](#)

| Name ?     | Type      | Default Value ? | Primary                             |
|------------|-----------|-----------------|-------------------------------------|
| id         | # int8    | NULL            | <input checked="" type="checkbox"/> |
| user_id    | uuid      | auth.uid()      | <input type="checkbox"/>            |
| filename   | T text    | NULL            | <input type="checkbox"/>            |
| url        | T text    | NULL            | <input type="checkbox"/>            |
| created_at | timestamp | now()           | <input type="checkbox"/>            |

[Add column](#)

Foreign keys

docs\_user\_id\_fkey

Foreign key relation to: [auth.users](#) [Edit](#) [Remove](#)

user\_id → auth.users.id

*Supabase bucket :*

Storage

[+ New bucket](#)

ALL BUCKETS

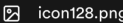
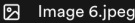
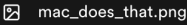
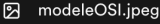
[docBucket](#)

CONFIGURATION

[Policies](#)

[Settings](#)

< 815781be-1992-48b6-9606-69f014351696

|                            |                                                                                      |
|----------------------------|--------------------------------------------------------------------------------------|
| 815781be-1992-48b6-9606... |  |
| b9bc8458-aad1-42c1-8865... |  |
|                            |  |
|                            |  |
|                            |  |
|                            |  |
|                            |  |